Notes: Assignment and Increment Operators

Programs change their variables all the time. A score goes up, lives go down, a total keeps growing. These notes cover the operators C# gives you for changing a variable: plain assignment `=`, the compound assignment shortcuts like `+=`, and the increment and decrement operators `++` and `--`.

1. The assignment operator `=`

```
int score = 10;
score = 25;
```

A single equals sign is the **assignment operator**. Read it as **"gets"**: "score gets 25."

It does **not** mean "is equal to" the way it does in math. It's an instruction: work out the value on the right, then store it in the variable on the left, replacing whatever was there.

The right side always goes first

```
int score = 10;
score = score + 5;
```

In math, `score = score + 5` makes no sense. In C#, it's normal:

1. C# works out the right side first: `score + 5` is `10 + 5`, which is `15`.
2. Then it stores that answer in `score`.

So `score` is now `15`. The old value was used to make the new one.

The left side must be a variable

The left side is where the answer goes, so it must be a variable. `10 = score;` won't build.

Don't mix up `=` and `==`

Operator	Name	Meaning	Example
`=`	Assignment	Store a value ("gets")	`score = 15;`
`==`	Equality	Ask whether two values are equal (gives `true` or `false`)	`score == 15`

2. Compound assignment operators

Changing a variable using its own value is so common that C# has a shortcut for it. Instead of writing the variable's name twice, you put the arithmetic operator in front of the `=`.

Shortcut	Means the same as	Read it as
`x += 5;`	`x = x + 5;`	add 5 to x
`x -= 5;`	`x = x - 5;`	take 5 away from x
`x *= 5;`	`x = x * 5;`	multiply x by 5
`x /= 5;`	`x = x / 5;`	divide x by 5
`x %= 5;`	`x = x % 5;`	replace x with the remainder of x divided by 5

Example, tracking `score` line by line:

```
int score = 10;
score += 5;   // 15
score -= 3;   // 12
score *= 2;   // 24
score /= 5;   // 4  (24 / 5 is 4.8, but score is an int, so it chops)
score %= 3;   // 1  (4 divided by 3 is 1, remainder 1)
```

The shortcuts don't do anything new. They're just shorter to write and easier to read, and you only type the variable's name once, so there's less chance of a typo.

Rule 1: the right side is worked out first, as a whole

```
int x = 4;
x *= 2 + 3;
```

This is **not** `x * 2 + 3` (which would be 11). It means `x = x * (2 + 3)`. The whole right side is worked out first, then the operator is applied. So `x` becomes `4 * 5`, which is `20`.

Rule 2: the operator comes first, then the equals sign

```
int lives = 3;
lives -= 1;   // lives is 2: takes one away
lives =- 1;   // lives is -1: just assigns negative 1
```

`-=` (minus-equals) takes something away. `=-` (equals-minus) is read by C# as `lives = -1`. It builds, it runs, and C# doesn't warn you. It's just wrong. The same goes for `=+`, which just assigns a positive number.

Remember: **operator first, then equals.**

Rule 3: no space inside the operator

`+=` is one operator made of two characters, so they must be touching. `score + = 5;` won't build (error CS1525, "Invalid expression term '='").

Rule 4: the usual type rules still apply

The shortcuts follow all the same rules as the long form.

Integer division still chops:

```
int total = 17;
total /= 5;   // 3, not 3.4
```

A double won't go into an int:

```
int total = 10;
total += 2.5;   // won't build
```

The answer would be `12.5`, a double, and a double won't fit in an int without a cast. The message is error CS0266, "Cannot implicitly convert type 'double' to 'int'". If you need decimals, make the variable a `double`:

```
double price = 20;
price *= 1.15;   // 23
double half = 10;
half /= 4;       // 2.5
```

(On a computer set to French (Canada), that last one prints as `2,5`.)

`+=` joins strings:

```
string message = "Hi";
message += " there";   // "Hi there"
message += 5;          // "Hi there5"
```

When the variable is a string, `+=` adds text to the end of it.

Rule 5: the variable needs a value first

```
int count;
count += 1;   // won't build
```

`count += 1` means `count = count + 1`, and C# can't add 1 to a value that doesn't exist yet. The message is error CS0165, "Use of unassigned local variable 'count'". Give it a starting value: `int count = 0;`

3. Increment and decrement: `++` and `--`

Adding or taking away exactly **one** is so common that it has its own operators.

Operator	Name	Means the same as
`x++;`	Increment	`x += 1;` or `x = x + 1;`
`x--;`	Decrement	`x -= 1;` or `x = x - 1;`

```
int lives = 3;
lives--;   // 2
lives--;   // 1
lives++;   // 2
```

They only ever change the variable by exactly one. To change it by any other amount, use `+=` or `-=`.

In this course: on a line of their own

You'll sometimes see `++` and `--` used inside a bigger expression, and written in two ways: before the variable (`++x`, called **prefix**) and after it (`x++`, called **postfix**). Inside an expression, the two give **different** values:

```
int m = 5;
Console.WriteLine(m++);   // prints 5, then m becomes 6

int k = 5;
Console.WriteLine(++k);   // k becomes 6, then prints 6
```

This is a common source of bugs, even for experienced programmers. So in this course, **`++` and `--` always go on a line of their own**, where both forms do exactly the same thing:

```
lives++;
Console.WriteLine(lives);
```

4. Common errors

What you see	Why	Fix
A variable is negative when you expected it to go down by a bit	Wrote `=-` instead of `-=`	Operator first: `-=`
`x *= 2 + 3` gives 20, not 11	The right side is worked out first	That's how it works. Use the long form if you mean `x * 2 + 3`.
`/=` lost the decimals	An int divided by an int still chops	Use a `double` if you need decimals
`score + = 5;` won't build	A space inside the operator	No space: `+=`
`total += 2.5;` won't build	A double won't go into an int	Make `total` a `double`
`count += 1;` won't build	The variable has no starting value	Give it one: `int count = 0;`
`if (score = 10)` won't build	`=` assigns; it doesn't compare	Use `==`

5. Check yourself

Track `p` through every line. Write its value after each one, then check your answers below.

```
int p = 6;
p += 9;         // 1
p -= 4;         // 2
p *= 3;         // 3
p /= 4;         // 4
p %= 5;         // 5
p++;            // 6
p *= 2 - 1;     // 7
p--;            // 8
p =+ 2;         // 9
```

10. What does `word` hold at the end?

```
string word = "Go";
word += "al";
word += 2 * 3;
```

Answers

#	Value of `p`	Why
1	15	6 + 9
2	11	15 - 4
3	33	11 × 3
4	8	33 / 4 is 8.25, and an int chops it to 8
5	3	8 divided by 5 is 1, remainder 3
6	4	`++` adds exactly one
7	4	The right side first: 2 - 1 is 1, and 4 × 1 is 4
8	3	`--` takes away exactly one
9	2	The trap: `=+` isn't `+=`. It just assigns positive 2.

10. `"Goal6"`. `"Go"` plus `"al"` is `"Goal"`. Then the right side is worked out first: `2 * 3` is `6`, and `+=` on a string adds it to the end.